

# FEATURE SPECIFICATIONS DOCUMENT

## 1. FEATURE OVERVIEW

**Feature Name:** AI Wardrobe Scanning & Auto-Tagging

**Feature Category:** Core Feature

**Priority Level:** Must-Have

**Target Release:** v1.0 (MVP Launch)

**Problem Statement:**

Users struggle to digitize their wardrobe manually. Existing apps require labor-intensive item entry, causing abandonment during onboarding. To unlock ClosetClear's value (outfits, resale, donations, analytics), the system must accurately scan clothing with minimal user effort.

**Success Criteria:**

- 95%+ tagging accuracy for category (shirt, dress, blazer, etc.)
- 90%+ color detection accuracy
- 80%+ brand/logo recognition accuracy (when visible)
- Average scan time: <5 seconds per item
- User onboarding completion increases to 75%+

**User Impact:**

- Faster onboarding and lower abandonment rates
- Increased engagement due to accurate wardrobe dataset
- Improved quality of recommendations, resale pricing, and analytics

---

## 2. USER PERSONAS & USE CASES

**Primary Persona: Urban Professional (Age 22–40)**

- **Demographics:** Busy schedule, mid-high disposable income, tech savvy
- **Goals:**
  - Digitize closet quickly
  - Optimize what to wear
  - Reduce clutter
- **Pain Points:**
  - "I don't have time to manually add 200+ items."
  - "I forget what clothes I own."
- **Context:**

- Uses the feature during onboarding, in bedroom or closet
- Uses mobile phone camera (iOS/Android)

## Secondary Persona: Fashion Enthusiast

- **Demographics:** Trend-focused, high engagement user, strong interest in detail
- **Goals:**
  - Get accurate tagging for materials, prints, fashion attributes
  - Create sharable closet content
- **Use Case Differences:**
  - Will rescan items for better accuracy
  - Wants deeper metadata (season, aesthetic, style type)

## Edge Cases

- Poor lighting or cluttered backgrounds
- Oversized items or non-standard clothing shapes
- Multi-piece outfits (e.g., suits, sets)
- Items on hangers vs flat-lay vs worn by user

---

# 3. DETAILED USER STORIES

## Epic

As a user, I want to scan my clothing and automatically generate item details so I can build a digital wardrobe without manual data entry.

---

### Story 1: Core Functionality – Single Item Scan

As a busy user

**I want** to scan an item of clothing using my phone camera

**So that** the system can detect its type, color, brand, material, and condition automatically.

#### Acceptance Criteria

- **Given** the user is on the scan screen  
**When** they capture or upload a photo  
**Then** the system identifies category, color, pattern, brand (if visible), and estimated condition.
  - **Given** lighting or background is poor  
**When** the system processes the image  
**Then** user is prompted to adjust lighting or background if accuracy confidence is low.
-

## Story 2: Error Handling & Metadata Correction

**As a user**

**I want** to quickly correct inaccurate tags

**So that** my wardrobe stays accurate.

### Acceptance Criteria

- Errors trigger actionable prompts (e.g., “Is this a T-shirt or Tank Top?”)
  - The corrected data is immediately saved
  - The AI model uses corrections to improve future scans
  - User never loses data if app crashes mid-scan
- 

## Story 3: User Experience – Batch Scan

**As a user**

**I want** to scan multiple items quickly

**So that** I can digitize my wardrobe in one session.

### Acceptance Criteria

- User can take multiple photos in sequence
  - System queues images and processes them asynchronously
  - Loading states communicate progress
  - Batch scan results appear in a summary review screen
- 

# 4. FUNCTIONAL REQUIREMENTS

## Core Functionality

---

### Function 1: Image Capture & Pre-Processing

- **Input:** User captures a photo or uploads an image
- **Processing:**
  - Background removal
  - Object detection (bounding box around garment)
  - Lighting normalization
- **Output:** Cleaned image passed to the recognition model
- **Validation:**
  - Minimum resolution 720p
  - Clothing object must be centered
  - Alert user if no identifiable item is detected

---

## Function 2: Clothing Recognition Engine

- **Input:** Pre-processed image
- **Processing:** Model identifies:
  - Category (shirt, dress, pants)
  - Subcategory (v-neck, crewneck, slim-fit)
  - Color(s)
  - Material (cotton, denim, wool)
  - Pattern (striped, floral, solid)
  - Brand detection via logo recognition
  - Condition estimation
- **Output:** JSON metadata object
- **Edge Cases:**
  - No logos detected → brand = "Unknown"
  - Multi-color items → color array
  - Blurry images → prompt to recapture

---

## Business Rules

- Category is mandatory; all other attributes optional
- User can always override AI results
- Each scan is versioned for future retraining
- Confidence score < 50% triggers validation prompt

---

## Integration Requirements

- Works with **User Wardrobe Database**
- Syncs with **Outfit Recommendation Engine**
- Syncs with **Resale Valuation Engine**
- Optional integration with **on-device model (Apple CoreML/TensorFlow Lite)**

---

# 5. NON-FUNCTIONAL REQUIREMENTS

## Performance

- Response time: <3 seconds (on-device), <5 seconds (cloud)
- Support up to 200 items per user
- Scalable for 1M+ scans per month

## Security

- On-device image processing where possible
- No storage of raw photos unless user consents
- Encrypted metadata in transit and at rest
- Audit logs for every metadata change

## Usability

- WCAG AA compliance
  - Supports latest 2 versions of iOS & Android
  - Works in low bandwidth mode
  - Tutorial on first use
- 

# 6. TECHNICAL SPECIFICATIONS

## Frontend Requirements

- Components:
    - Camera module
    - Scan progress modal
    - Metadata review form
    - Batch scan carousel
  - State Management:
    - React Native context or Redux for scan state
  - Routing:
    - /scan
    - /scan/review
  - Styling:
    - Use Tailwind + design system tokens
- 

## Backend Requirements

### API Endpoints

#### POST /api/scan

- Request body:

```
{
  "image": "<base64>",
  "userId": "string"
}
```

- Response:

```
{
  "category": "...",
  "color": ["..."],
}
```

```
"brand": "...",
"material": "...",
"pattern": "...",
"condition": "...",
"confidence": {
  "category": 0.92,
  "brand": 0.48
}
}
```

- Validation:
  - Reject images over 5MB
  - Reject unsupported formats

---

## Database Changes

### New Table: wardrobe\_items

Fields:

- item\_id (PK)
- user\_id (FK)
- image\_url
- category
- attributes (JSONB)
- brand
- condition
- created\_at
- updated\_at

Indexes:

- idx\_user\_id
- idx\_category
- idx\_brand

---

## 7. USER INTERFACE SPECIFICATIONS

### Layout Requirements

- Full-screen camera view
- Bottom sheet for instructions
- Review screen with editable tags

### Interactive Elements

- Primary button: “Scan Item”
- Secondary button: “Add Another”

- Edit chips for tags
- Loading skeletons for AI analysis

## **Responsive Design**

- Mobile-first
  - iPad/tablet landscape support
  - Compress image sizes dynamically
- 

# **8. TESTING REQUIREMENTS**

## **Unit Tests**

- Tag extraction logic
- Confidence score thresholds
- Failure states (null image, corrupted image)

## **Integration Tests**

- Camera → preprocessing → AI inference → DB save
- Metadata corrections
- Batch scan workflow

## **User Acceptance Tests**

- Scenario 1: First-time single item scan
- Scenario 2: Batch scan of 5 items
- Scenario 3: Incorrect category correction

## **Performance Tests**

- 1000 concurrent scans
  - Poor network conditions
  - Stress test: 200 items in a batch
- 

# **9. IMPLEMENTATION PLAN**

## **Development Phases**

### **Phase 1 (5 days): Core System**

- Camera module
- Image upload pipeline
- Simple AI model integration

## Phase 2 (7 days): Metadata & UI

- Review screen
- Tag editing
- Batch workflow

## Phase 3 (5 days): QA + Optimization

- Model tuning
  - Performance refinement
  - UX polish
- 

## Dependencies

- AI model readiness
- Device camera permissions
- Cloud inference setup

## Risk Assessment

- **Technical Risk:** Brand/logo recognition accuracy
  - **Timeline Risk:** Cloud inference delays
  - **Mitigation:** Fallback to on-device light model
- 

# 10. SUCCESS METRICS & MONITORING

## Feature Adoption

- Scan completion rate
- Onboarding drop-off reduction
- Number of items scanned per user

## Technical Metrics

- Inference speed
- Accuracy scores
- Cloud vs on-device distribution

## Business Metrics

- Conversion from free → paid (tagging quality)?
- Increase in resale listing generation
- Increase in outfit recommendation usage

## Monitoring Setup

- Sentry error logging
- Firebase Analytics events
- CloudWatch inference logs